

BTS SIO - OPTION SISR

Module : Support et Mise à Disposition de Services

La Conteneurisation avec Docker

De la théorie aux cas pratiques (Nginx, Portainer, GLPI)



Par : Mejendel Mondher

Sommaire

- **1. Introduction : Conteneurisation vs Virtualisation**
- **2. Comment fonctionnent les conteneurs sous le capot ?**
- **3. Atelier : Installation de Docker Engine (Ubuntu/Debian)**
- **4. Écrire et construire des images de conteneurs**
- **5. Outil de Gestion : Portainer**
- **6. Cas Pratique n°1 : Serveur Web Nginx**
- **7. Cas Pratique n°2 : Gestion de parc avec GLPI & MariaDB**
- **8. Conclusion**

1. Introduction : Conteneurisation vs Virtualisation

Jusqu'à présent, pour isoler des services, vous avez utilisé la **virtualisation** (via Proxmox ou VMware). Cette méthode est excellente pour tester plusieurs versions d'un OS, exécuter des applications "legacy" ou isoler des environnements avec une très forte exigence de sécurité. Mais ce niveau d'isolation a un coût : les Machines Virtuelles (VMs) sont lourdes, lentes à démarrer et consomment beaucoup de ressources matérielles car elles émulent un OS complet.

La conteneurisation répond à une toute autre logique : une isolation à l'échelle de l'application.

La conteneurisation ne cherche pas à émuler un système entier. **Elle vise à isoler uniquement l'application et ce dont elle a besoin pour fonctionner (code, dépendances, configurations).** Les conteneurs partagent le noyau (kernel) du système hôte, **ce qui les rend infiniment plus légers et rapides.**

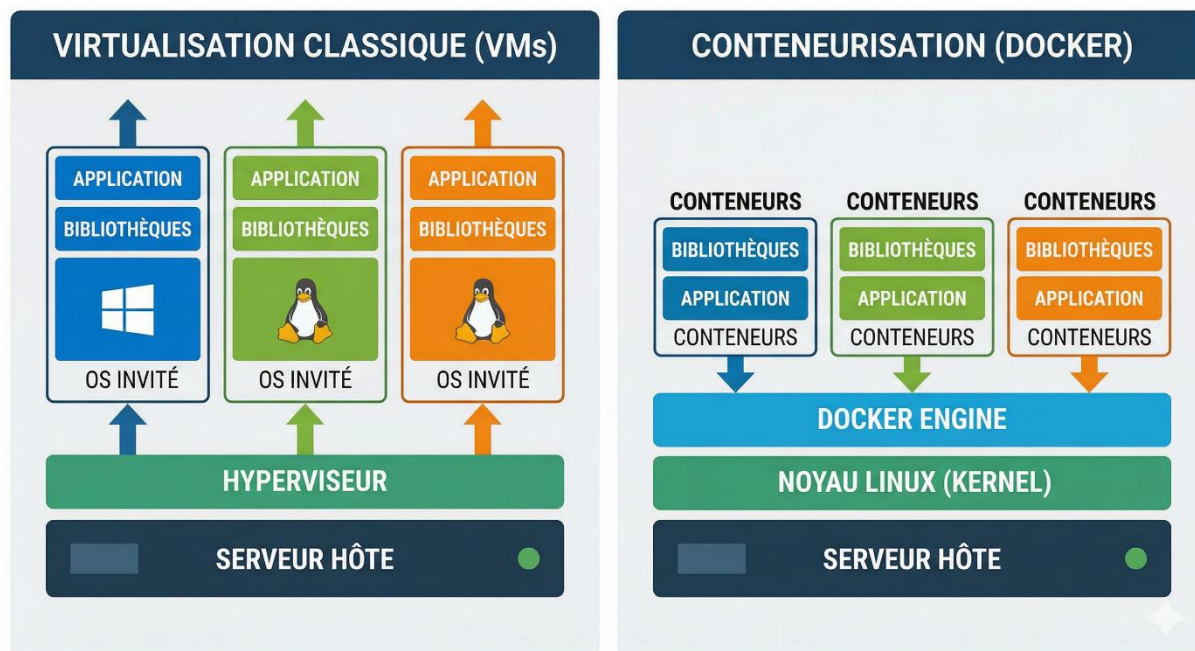


Tableau Comparatif

Critère	Virtualisation (VMs)	Conteneurisation (Docker)
Isolation	OS complet isolé via Hyperviseur	Processus isolé, noyau hôte partagé
Taille	Plusieurs Go	Quelques Mo à centaines de Mo
Démarrage	Minutes (Boot de l'OS)	Secondes (Lancement du processus)
Ressources	Élevées (RAM et CPU dédiés)	Légères (Partage dynamique des ressources)
Portabilité	Dépend de l'hyperviseur	Très portable (Standard OCI)

Les 6 avantages clés de la conteneurisation

- **Portabilité et cohérence** : L'environnement est encapsulé. Fini le syndrome du "Ça marche chez moi mais pas en production".
- **Efficacité des ressources** : Partageant le noyau de l'hôte, on peut faire tourner beaucoup plus d'applications sur un même serveur.
- **Déploiement ultra-rapide** : Création, arrêt et suppression se font en quelques secondes.
- **Isolation renforcée** : Cloisonnement des processus pour limiter l'impact d'un dysfonctionnement.
- **Scalabilité dynamique** : Ajustement automatique du nombre de conteneurs selon la charge du réseau.
- **Développement simplifié** : Idéal pour l'intégration continue (CI/CD).

2. Comment fonctionnent les conteneurs sous le capot ?

Il n'y a pas de "magie" dans Docker. Son efficacité repose sur l'exploitation de trois fonctionnalités fondamentales et natives du noyau Linux. Ces trois piliers travaillent ensemble pour créer l'illusion d'une machine indépendante :

1. Isolation avec les Namespaces

Les namespaces sont des mécanismes du noyau Linux qui créent des environnements isolés. Chaque namespace offre au conteneur une vue distincte et cloisonnée des ressources du système. Par exemple :

- **Namespace PID** : Le conteneur possède son propre arbre de processus (il aura son propre PID 1, indépendant de l'hôte).
- **Namespace Réseau (NET)** : Offre à chaque conteneur sa propre pile réseau (interfaces, règles de pare-feu, IP).
- **Namespace Utilisateur (USER)** : Isole les UID/GID.

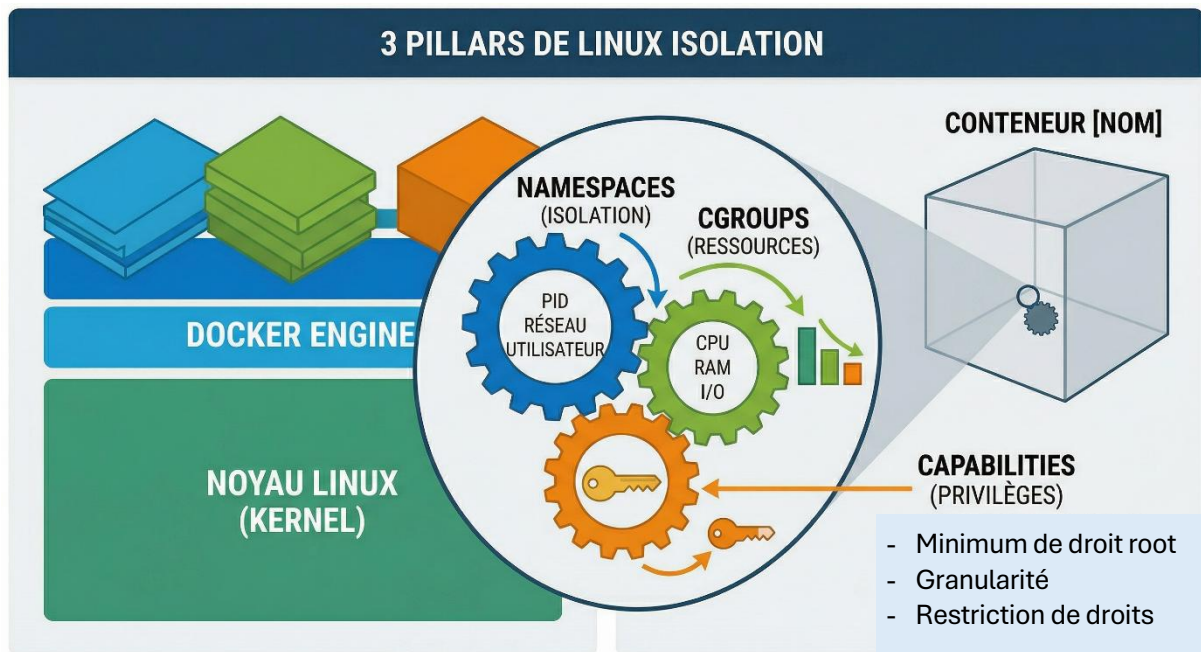
Résultat : Le processus du conteneur ne peut pas voir ni affecter les processus de l'hôte ou des autres conteneurs.

2. Gestion des privilèges avec les Capabilités (Capabilities)

Traditionnellement, l'utilisateur "root" a tous les droits sur Linux. Les capacités fragmentent ces privilèges immenses en unités plus petites. Cela permet d'attribuer à un processus de conteneur *uniquement* les droits dont il a besoin (ex: modifier l'heure, ouvrir un port réseau inférieur à 1024) sans lui donner un accès root global. Cela réduit drastiquement les risques en cas de compromission (piratage) du conteneur.

3. Allocation des ressources avec les Cgroups (Control Groups)

Les cgroups permettent de limiter, hiérarchiser et surveiller l'utilisation matérielle. C'est grâce aux cgroups que nous pouvons définir qu'un conteneur web ne doit pas consommer plus de 512 Mo de RAM ou plus de 20% du CPU. Cela garantit qu'un conteneur défaillant ne monopolisera pas toutes les ressources du serveur physique.



3. Installation de Docker Engine

Prérequis : Une machine virtuelle sous Ubuntu Server ou Debian, connectée à Internet.

Plusieurs possibilités existent quant à la méthode d'installation de Docker Engine - le moteur principal de Docker - à savoir :

- Utiliser **Docker Desktop** pour Linux (plus simple)
- Configurer et installer Docker via le dépôt **apt**
- Installer (et mettre à jour) manuellement avec un **fichier de package deb**
- Utiliser un **script** pratique (mis à disposition par Docker)

Contrairement à une application classique, Docker interagit profondément avec le noyau Linux. Nous allons l'installer via le **script officiel fourni par l'éditeur**, qui se charge de configurer les dépôts et les clés de sécurité automatiquement.

Étape 1 : Mise à jour du système

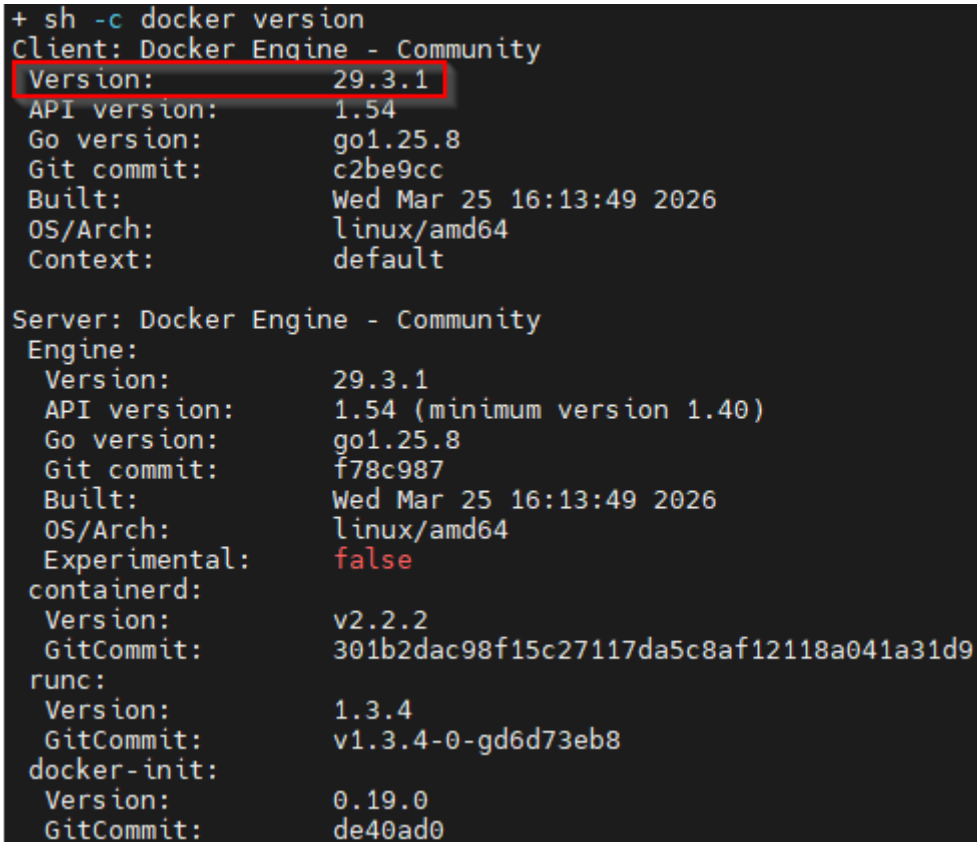
Il est toujours indispensable de mettre à jour la liste des paquets avant une installation :

```
sudo apt update && apt upgrade -y
```

Étape 2 : Lancement du script d'installation Docker

Nous allons télécharger le script avec `curl` et l'exécuter directement :

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

A terminal window showing the output of the 'docker version' command. The output is divided into two sections: 'Client: Docker Engine - Community' and 'Server: Docker Engine - Community'. The 'Client' section lists: Version: 29.3.1 (highlighted with a red box), API version: 1.54, Go version: go1.25.8, Git commit: c2be9cc, Built: Wed Mar 25 16:13:49 2026, OS/Arch: linux/amd64, and Context: default. The 'Server' section lists: Engine: Version: 29.3.1, API version: 1.54 (minimum version 1.40), Go version: go1.25.8, Git commit: f78c987, Built: Wed Mar 25 16:13:49 2026, OS/Arch: linux/amd64, Experimental: false, containerd: Version: v2.2.2, GitCommit: 301b2dac98f15c27117da5c8af12118a041a31d9, runc: Version: 1.3.4, GitCommit: v1.3.4-0-gd6d73eb8, docker-init: Version: 0.19.0, and GitCommit: de40ad0.

```
+ sh -c docker version
Client: Docker Engine - Community
Version: 29.3.1
API version: 1.54
Go version: go1.25.8
Git commit: c2be9cc
Built: Wed Mar 25 16:13:49 2026
OS/Arch: linux/amd64
Context: default

Server: Docker Engine - Community
Engine:
Version: 29.3.1
API version: 1.54 (minimum version 1.40)
Go version: go1.25.8
Git commit: f78c987
Built: Wed Mar 25 16:13:49 2026
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: v2.2.2
GitCommit: 301b2dac98f15c27117da5c8af12118a041a31d9
runc:
Version: 1.3.4
GitCommit: v1.3.4-0-gd6d73eb8
docker-init:
Version: 0.19.0
GitCommit: de40ad0
```

Ce script permet aussi de voir la version installée. Vous pouvez aussi taper la commande : `docker version`

Étape 3 : Post-installation (Étape critique)

Par défaut, seul l'utilisateur **root** a le droit de communiquer avec le moteur Docker. Si vous tapez `docker ps` avec votre utilisateur standard, vous obtiendrez une erreur "Permission denied".

Pour éviter de devoir taper `sudo` devant chaque commande (ce qui est une mauvaise pratique de sécurité), nous allons ajouter votre utilisateur actuel (`$USER`) au groupe système `docker` :

```
sudo usermod -aG docker $USER
```

⚠ **Attention : Pour que ce changement de groupe soit pris en compte par Linux, vous devez vous déconnecter de votre session (tapez `exit`) et vous reconnecter à votre serveur !**

Étape 4 : Vérification

Une fois reconnecté, vérifiez que le service est actif :

```
docker --version
docker system info
```

4. Écrire et construire des images de conteneurs

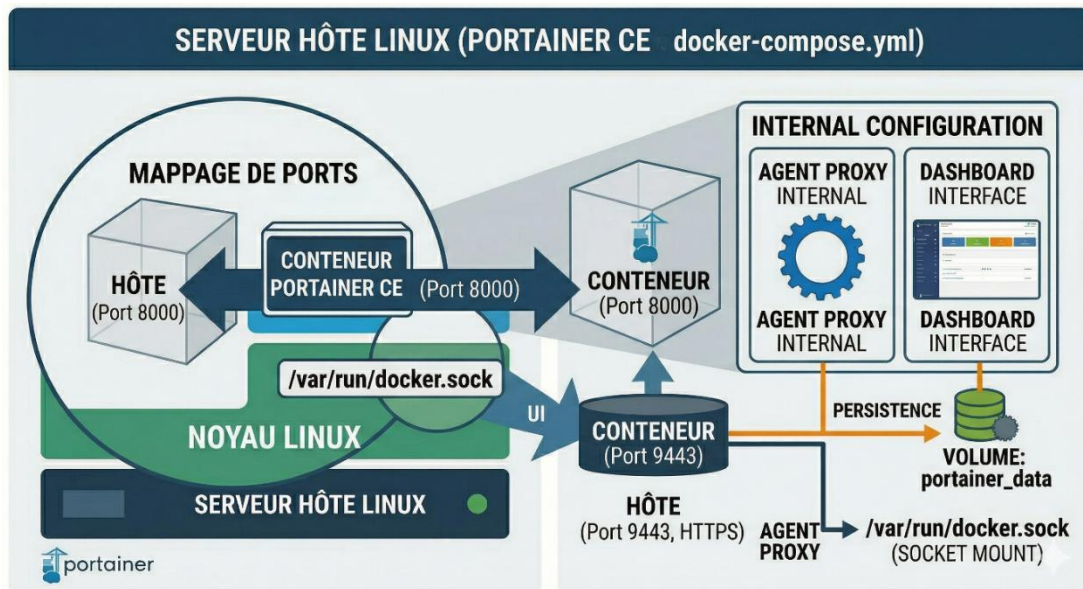
Une image de conteneur est une photographie figée (un modèle en lecture seule) prête à être lancée. Elle contient le code, les dépendances, et la configuration. Ces images sont construites à partir d'un fichier recette : le **Dockerfile**.

Le fonctionnement en couches (Layers) :

Chaque instruction du Dockerfile crée une couche dans l'image. Ces couches sont empilées et *mises en cache*. Si vous modifiez une ligne de code, seule la dernière couche est reconstruite.

5. Outil de Gestion : Portainer

Gérer de nombreux conteneurs en ligne de commande peut devenir fastidieux. **Portainer** est une interface Web d'administration de conteneurs très utilisée en entreprise.



Installation de Portainer

Sur votre serveur Linux hôte, exécutez ces commandes :

```
docker volume create portainer_data

docker run -d -p 8000:8000 -p 9443:9443 --name portainer \
  --restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data \
  portainer/portainer-ce:latest
```

Explication : Le montage `-v /var/run/docker.sock:/var/run/docker.sock` permet au conteneur Portainer de piloter le moteur Docker de la machine hôte.

Accédez ensuite à **[https://\[IP_DE_VOTRE_SERVEUR\]:9443](https://[IP_DE_VOTRE_SERVEUR]:9443)** (Acceptez l'alerte de sécurité SSL) pour **initialiser le mot de passe administrateur**.

PORTAINER.io

✓ New Portainer installation

Please create the initial administrator user.

Username

admin



Password

.....

Confirm password

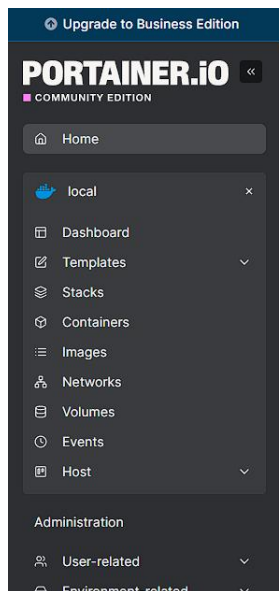
.....



⚠ The password must be at least 12 characters long.

Create user

> Restore Portainer from backup



Environments

Home

admin

Latest News From Portainer

Docker v29.3.0 fixes a critical CVE but also has breaking changes, especially for Swarm users. Learn more about [the CVE](#) and the [breaking changes](#) in our blogs.

Environments

Search by name, group, tag, status, URL...

Refresh

Kubeconfig

Click on an environment to manage

Platform

Conne...

Status

Tags

Groups

Agent ...

Clear all

Sort ...

↓↑



local

Up

2026-04-07 14:20:15

Standalone 29.3.1

/var/run/docker.sock

Group: Unassigned

No tags

Local

0 stacks

2 containers

1

1

0

0

0

1 volume

3 images

2 CPU

2.1 GB RAM

Disconnect

Connected

Items per page

10

6. Cas Pratique n°1 : Serveur Web Nginx Haute Performance

Objectif : Déployer un serveur web en ligne de commande, comprendre le mappage de ports et nettoyer l'environnement.

Étape 1 : Lancement du service

```
docker run -d --name serveur-nginx -p 8080:80 nginx:latest
```

Explications :

- `-d` : Detached mode (tourne en arrière-plan).
- `-p 8080:80` : Règle NAT. Le trafic arrivant sur le port 8080 de l'hôte est redirigé vers le port 80 du conteneur.

Étape 2 : Vérification et Monitoring

```
# Voir les conteneurs actifs
```

```
docker ps
```

```
# Voir les statistiques de consommation (preuve des Cgroups)
```

```
docker stats serveur-nginx
```

**Testez le bon fonctionnement en allant sur
`http://[IP_DE_VOTRE_SERVEUR]:8080`.**



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.

To engage with the community please visit community.nginx.org.

For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

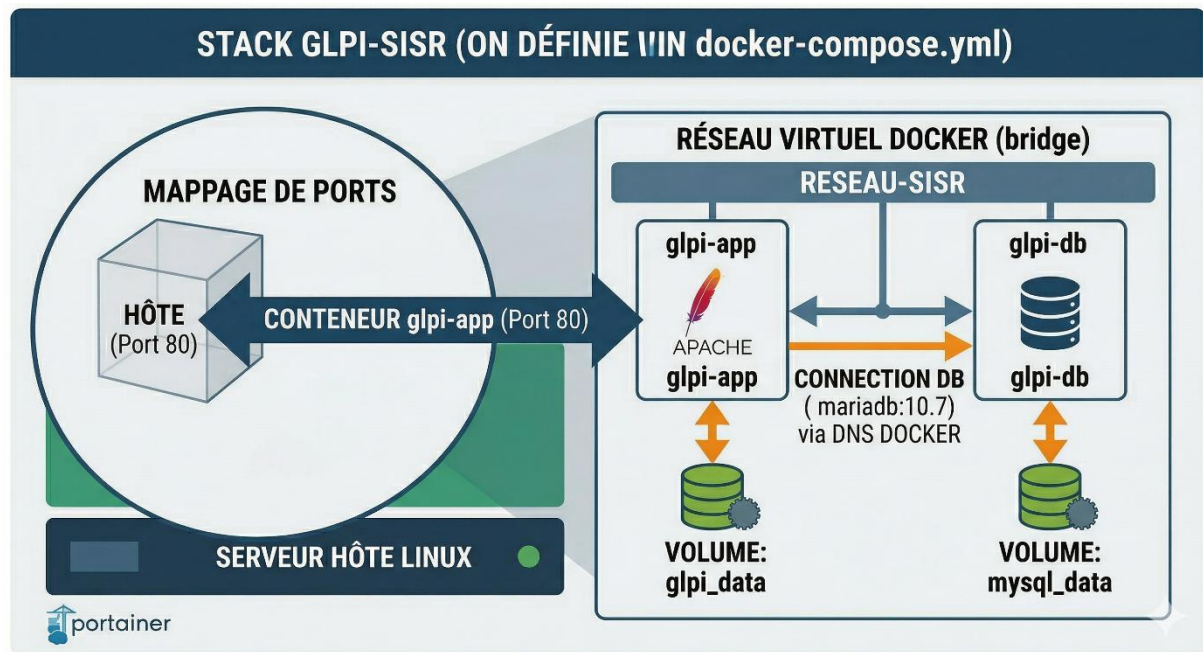
Thank you for using nginx.

Étape 3 : Arrêt et Nettoyage

```
docker stop serveur-nginx
docker rm serveur-nginx
```

7. Cas Pratique n°2 : Gestion de parc avec GLPI & MariaDB

Objectif : Déployer une application métier avec sa base de données via Portainer (Docker Compose).



1. Préparation du fichier Compose

```
version: "3.2"
services:
  mariadb:
    image: mariadb:10.7
    container_name: glpi-db
    environment:
      MYSQL_ROOT_PASSWORD: Poiuytreza-75
      MYSQL_DATABASE: glpidb
      MYSQL_USER: glpi_user
      MYSQL_PASSWORD: glpi_password
    volumes:
      - mysql_data:/var/lib/mysql
    restart: always

  glpi:
    image: diouxx/glpi
    container_name: glpi-app
    ports:
      - "80:80"
    environment:
      TIMEZONE: 'Europe/Paris'
    volumes:
      - glpi_data:/var/www/html/glpi
    depends_on:
      - mariadb
    restart: always

volumes:
  mysql_data:
  glpi_data:
```

2. Déploiement via Portainer

1. Dans Portainer, allez dans **Stacks** > **+ Add stack**.
2. Nommez votre stack en **lettres minuscules** (ex: parc-glpi).
3. Collez le code YAML et cliquez sur **Deploy the stack**.

Connectez-vous sur `http://[IP_DE_VOTRE_SERVEUR]` pour installer GLPI.

- Serveur SQL : mariadb (Résolution DNS Docker !)

- Utilisateur : `glpi_user` / Mot de passe : `glpi_password` / Base : `glpidb`

8. Conclusion

Vous êtes désormais capables de justifier le choix entre VM et Conteneur, d'installer un moteur Docker, de déployer des services réseau quasi-instantanément et d'orchestrer des applications multi-services avec Portainer !

Liens utiles :

<https://blog.stephane-robert.info/docs/conteneurisation/>

<https://www.it-connect.fr/cours/docker-et-kubernetes-initiation-a-la-mise-en-production/>